NETWORKING

GraphQL Visual Guide

Ask for exactly what you need — schemas, resolvers, and the N+1 trap.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/5

Schema & Types: The Contract

A typed graph of everything clients may ask for.

```
type User { id: ID!  name: String!  posts: [Post!]! }

type Post { id: ID!  title: String!  author: User! }

type Query { user(id: ID!): User }

type Mutation { createPost(title: String!): Post }
```

- The schema is the API: strongly typed, introspectable — clients explore it like documentation.
- '!' means non-null; the type system validates every query BEFORE execution.
- One endpoint (/graphql); the query itself selects resources — no REST-style URL sprawl.

STEP 2/5

Queries & Mutations

Read with queries, write with mutations — shaped exactly as requested.

```
query { user(id: 7) { name  posts { title } } }
```

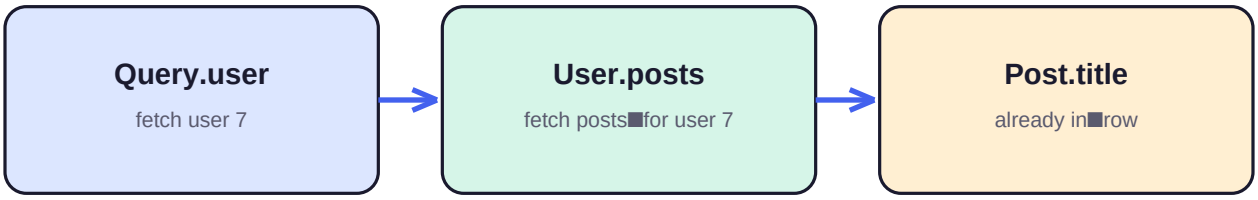
```
→ { "user": { "name": "Ada", "posts": [{"title": "…"}] } }
```

- No over-fetching: the response mirrors the query's field selection — mobile clients love this.
- No under-fetching: nested data in ONE round trip — the REST 'fetch user, then fetch posts' dance disappears.
- Mutations run serially (in order); queries run in parallel — the spec guarantees it.

STEP 3/5

Resolvers: Field-Level Functions

Every field is resolved by a function — the query walks the graph.

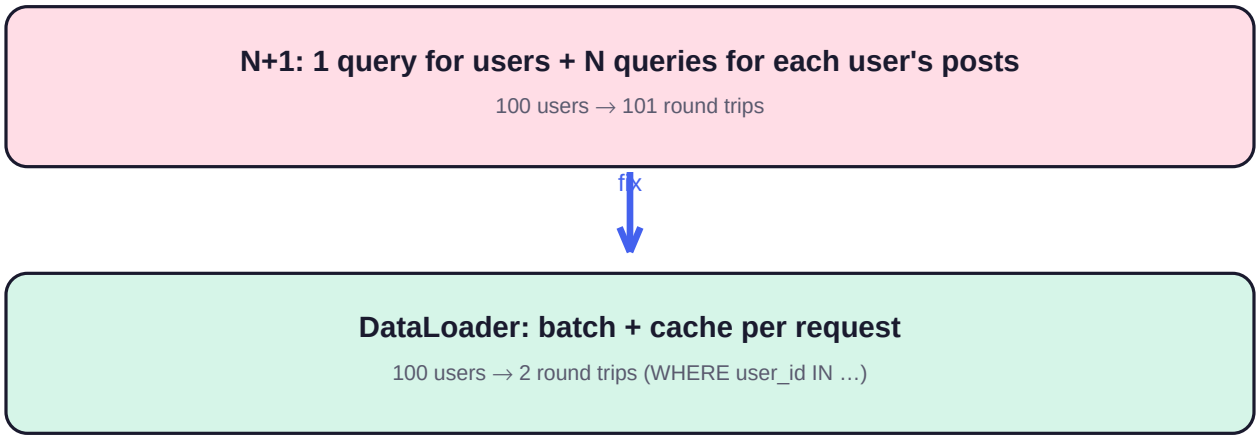


- Resolvers compose: parent results feed child resolvers — each level only fetches what was asked.
- Default resolvers just read properties; custom ones hit databases, services, caches.
- This elegance hides a performance trap — next page.

STEP 4/5

The N+1 Problem & DataLoader

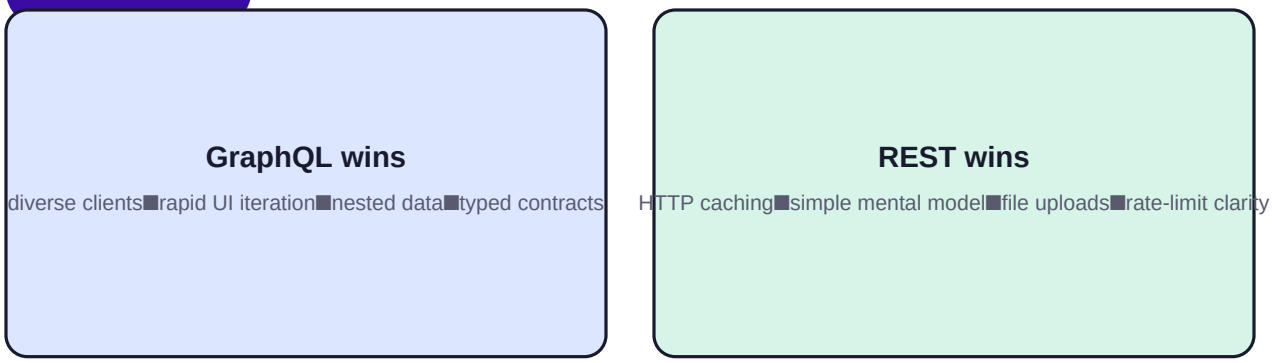
Naive resolvers query per row — batching fixes it.



- DataLoader collects keys during a tick, fires ONE batched query, and maps results back.
- Per-request cache: the same entity fetched twice in one query hits the cache, not the DB.
- Also cap query depth/complexity — malicious clients can craft exponentially expensive queries.

STEP 5/5

GraphQL vs REST



- Caching is harder: one endpoint + POST means HTTP caches don't help — plan persisted queries or app-level caching.
- Versioning: evolve the schema (deprecate fields) instead of /v1, /v2 — but removals still break clients.
- Many teams expose GraphQL as a gateway over REST/gRPC microservices.

STEP ✓I✓

Key Takeaways

GraphQL — Remember

- Schema = typed contract; introspection makes it self-documenting.
- Queries shape the response exactly — no over/under-fetching, one round trip.
- Resolvers compose field-by-field; watch for the N+1 trap.
- DataLoader batches and caches per request — 101 queries become 2.
- GraphQL for flexible clients; REST when HTTP caching and simplicity win.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev